

A.A. 2026-2027

Traccia Progetti TW e IS

Indice

0 Traccia Progetti TW e IS --- Side Quest 3

Formazione dei gruppi 3
Registrazione e identificativo 3
Scritto, progetto e propedeuticità 4
Architettura del sistema 4
Frontend 4
Backend 4
Scelta dello stack 5
Mappe, cloud e container 5
Il sistema Side Quest 5
Struttura dei requisiti 5
Ambiguità intenzionali 6
Assegnazione dei pacchetti 6
Modalità di consegna 6
Consegna intermedia 6
Output generali comuni 7
Tecnologie Web 7
Deliverable richiesti 7
Svolgimento della discussione 7
Criteri di valutazione 8
Ingegneria del Software 8
Analisi e specifica dei requisiti 8
System design 8
Object design 9
Analisi statica e versioning 9
Testing 9
Presentazione del progetto 9
Pesi di valutazione 10
Uso dell'intelligenza artificiale 10

Capitolo 0

Traccia Progetti TW e IS — Side Quest

L'A.A. 2026-2027 introduce un'unica traccia di progetto per Tecnologie Web e Ingegneria del Software: si sviluppa un solo sistema, producendo artefatti diversi che ciascun insegnamento valuta separatamente.

- I due esami restano **distinti**: due discussioni separate con criteri indipendenti; una valutazione alta in un esame non implica nulla sull'altra, perché i due corsi guardano ad aspetti diversi dello stesso progetto.
- È possibile sostenere anche un solo esame dei due, realizzando il medesimo sistema con i vincoli tecnologici previsti da quel solo insegnamento.
- Il documento della traccia (17 pagine) comprende una parte generale con le regole comuni, l'**Appendice A** con il sistema da realizzare e le appendici B (Tecnologie Web) e C (Ingegneria del Software) con gli output attesi.
- La traccia è volutamente complessa e la sua comprensione è parte della valutazione: va studiata con estrema attenzione prima di iniziare a lavorare.

Formazione dei gruppi

La composizione del gruppo segue regole precise su numerosità e vincoli di appartenenza.

- La numerosità ideale è **2 persone**; i gruppi da 3 sono ammessi in casi particolari.
- I gruppi da 1 richiedono un'autorizzazione esplicita (tipicamente studenti lavoratori) e sono fortemente sconsigliati: la componente di **teamwork**, valutata esplicitamente in Ingegneria del Software, non può essere valutata appieno.
- È fortemente consigliata l'**omogeneità**: tutti i membri dovrebbero sostenere gli stessi esami.

Info

Gruppi disomogenei

- Devono comunque produrre **tutti** gli output attesi per entrambi gli insegnamenti. - Tutti i membri devono conoscere l'intero progetto (architettura, requisiti, codice, scelte) e saper rispondere su qualsiasi parte in discussione. - Il lavoro viene comunque valutato come un gruppo da due.

Registrazione e identificativo

La registrazione avviene tramite un modulo online ufficiale.

- Il modulo richiede la numerosità del gruppo, l'insegnamento da sostenere e, per ogni membro, matricola, cognome, nome ed email istituzionale.
- Alla sottomissione arriva una mail automatica con un **ID gruppo progressivo** (es. `2627_001`), che determina le funzionalità assegnate.
- Ogni studente può appartenere a **un solo gruppo**: è vietato partecipare a gruppi diversi per i due insegnamenti, per evitare problemi di **authorship** e duplicazione di codice.
- In caso di dubbio, conviene indicare "progetto integrato": non obbliga a discutere entrambi gli esami.

- Chi non trova un compagno può usare i canali di ricerca creati nei Team dei corsi, contattando chi ha già postato prima di pubblicare un nuovo messaggio.
- Non esiste una scadenza per la formazione, ma le funzionalità sono assegnate solo dopo la registrazione.

Scritto, progetto e propedeuticità

Le prove d'esame e il progetto seguono regole di svincolo e validità precise.

- Progetto e prova scritta sono **completamente svincolati**: si può discutere il progetto prima di superare lo scritto e viceversa; per scritto si intende l'appello ordinario oppure l'esito complessivo delle due prove intercorso.
- Le propedeuticità non impediscono né lo scritto né la discussione del progetto, ma impediscono la **verbalizzazione** del voto.
- Le valutazioni e le consegne restano valide indicativamente un anno: fino al **31 marzo 2028**, termine dell'A.A. 2026-2027.
- Oltre quella data si passa alla traccia del nuovo anno accademico.

Info

Vecchi ordinamenti

- **TW 6 CFU (A.A. 2025-2026)**: resta valido il progetto dello scorso anno accademico, con validità estesa anche oltre marzo 2027; il progetto nuovo contiene lavoro che supera i 6 CFU, quindi resta facoltativo. - **IS 10 CFU**: le modalità dedicate verranno valutate e comunicate separatamente dai docenti.

Architettura del sistema

Il sistema è un'applicazione distribuita **client-server** composta da due macro-componenti che eseguono su processi o dispositivi separati e comunicano tramite **API REST** via rete.

- Il **backend** è responsabile della gestione dei dati persistenti: la persistenza deve risiedere interamente lato server, non nel browser (niente `localStorage` o variabili locali).
- Il **frontend** ha vincoli diversi a seconda degli esami da sostenere.

Frontend

I vincoli sul frontend dipendono dagli esami scelti dal gruppo.

- Per Tecnologie Web il frontend è obbligatoriamente una **Single Page Application**, sviluppata con framework come Angular, React, Vue o Svelte; Kotlin Multiplatform è ammesso se genera codice client JavaScript fruibile nel browser.
- L'SPA deve funzionare su qualsiasi browser, desktop e mobile.
- Se il gruppo sostiene solo Ingegneria del Software, il frontend non è vincolato alla SPA: sono ammesse interfacce desktop (Swing, JavaFX) o applicazioni mobile native (es. Android con Kotlin).

Backend

Il backend deve essere sviluppato **from scratch** con un framework di programmazione (Node.js, Express, Spring Boot, ecc.).

Warning

CMS e no-code vietati

Sono espressamente vietati i **Content Management System** (es. WordPress, Strapi) e le console **no-code/low-code** (es. creare API con due clic nella console Firebase): un backend non scritto dal gruppo impedisce ai docenti di valutare la capacità di progettare e implementare software da zero.

Scelta dello stack

Non esiste un elenco chiuso di tecnologie obbligatorie: la scelta dello stack è essa stessa un momento di maturazione e di valutazione.

- Lo stack deve permettere ai docenti di valutare le competenze del proprio corso: per IS contano modularità, coesione, accoppiamento e object design (un unico file senza orientamento agli oggetti non è valutabile); per TW contano reattività, sicurezza, usabilità ed efficienza.
- Le scelte di tecnologia devono quindi abilitare la valutazione, non ostacolarla.

Mappe, cloud e container

Per mappe, deployment ed esecuzione valgono i seguenti indirizzi.

- Le mappe interattive sono richieste: sono suggerite Leaflet con dati OpenStreetMap, ma sono ammesse anche Google Maps (previa registrazione di una API key), HERE o Apple Maps.
- Scostarsi dai suggerimenti è apprezzato, purché la scelta sia motivata (es. qualità dei dati cartografici).
- Il deployment su public cloud (AWS, Azure, GCP) è **apprezzato ma non richiesto**: i piani gratuiti sono sufficienti per il carico computazionale del progetto.
- Come soluzione minima consigliata è richiesta la **containerizzazione** tramite Docker e Docker Compose, per garantire un'esecuzione rapida e affidabile durante la valutazione.

Il sistema Side Quest

Definition

Side Quest

Piattaforma ludica per la creazione e lo svolgimento di "missioni" locali: ogni missione ha un reward in punti esperienza (XP), una posizione geografica sulla mappa e richiede il caricamento di una prova dello svolgimento (es. foto con timestamp).

- Esistono varianti previste dalla traccia: missioni a tempo, diversi tipi di account e altre descritte nei requisiti in linguaggio naturale.

Struttura dei requisiti

I requisiti funzionali sono 21, organizzati in un nucleo e in pacchetti.

- **Nucleo comune**: requisiti da 1 a 6, obbligatori per tutti i gruppi indipendentemente dalla numerosità.
- **Pacchetti aggiuntivi**: cinque pacchetti (A, B, C, D, E), ciascuno con 3 requisiti.
- Le funzionalità non assegnate al gruppo non sono obbligatorie e non vanno implementate.

Ambiguità intenzionali

La traccia contiene appositamente ambiguità e lacune di specifica.

- Ogni gruppo deve **individuare** le ambiguità e **risolverle autonomamente**, adottando scelte coerenti che non contraddicano i requisiti espliciti.
- Le decisioni prese vanno documentate e presentate in sede di discussione; non sono previste iterazioni di raffinamento con i docenti.

Assegnazione dei pacchetti

I pacchetti sono attribuiti in base all'ID del gruppo ricevuto alla registrazione.

- Pacchetto di partenza = (ultima cifra dell'ID gruppo) `mod 5`: resto 0 → pacchetto A, 1 → B, 2 → C, 3 → D, 4 → E.
- Se serve più di un pacchetto si prosegue in modo ciclico: A → B → C → D → E → A.
- Esempio: un ID che termina con 7 dà 7 mod 5 = 2, quindi si parte dal pacchetto C; in consegna ordinaria con due pacchetti si realizzano C e D.

Il perimetro funzionale dipende dalla dimensione del gruppo e dalla modalità di consegna.

Dimensione gruppo	Anticipata	Ordinaria
1	Nucleo comune (REQ 1–6)	Nucleo + primi 2 requisiti del primo pacchetto
2	Nucleo + 1 pacchetto	Nucleo + 2 pacchetti
3	Nucleo + 2 pacchetti	Nucleo + 3 pacchetti

Modalità di consegna

Esistono due modalità di svolgimento del progetto, che differiscono per perimetro funzionale e scadenze.

- **Modalità anticipata**: perimetro ridotto; richiede una consegna intermedia e la consegna e discussione della versione finale entro il 31 marzo 2027.
- **Modalità ordinaria**: perimetro completo; consegna e discussione possibili fino al 31 marzo 2028.
- Per ogni appello i docenti organizzano almeno una sessione di discussione, annunciata con avviso su Docenti/UNINA e nei Team dei corsi, con consegna richiesta alcuni giorni prima della sessione.
- Mancare la consegna intermedia non è un problema: si passa semplicemente alla modalità ordinaria con perimetro completo.

Consegna intermedia

La consegna intermedia è il requisito d'accesso alla modalità anticipata.

- Per **Ingegneria del Software** consiste nel solo documento di analisi e specifica dei requisiti (SRS), producibile subito con gli strumenti già visti a lezione.

- Per **Tecnologie Web** consiste nella sola componente backend dell'applicazione.
- Le scadenze orientative sono la prima settimana di dicembre per IS e una settimana dopo per TW.
- Quello che viene consegnato non è definitivo: può essere modificato senza penalizzazione, e aggiornare l'analisi dopo il design è anzi apprezzato come prova dell'interazione tra le fasi.

Output generali comuni

Ogni consegna, per entrambi gli insegnamenti, include gli stessi artefatti di base.

- **Codice sorgente** completo, analizzato in entrambe le discussioni (code review per TW; allineamento tra diagrammi e implementazione, e qualità generale, per IS).
- Applicazione **rieseeguibile** come la consegnerebbe un committente pagato: README con istruzioni di esecuzione e ambiente, script per popolare il database con dati di esempio.

Tecnologie Web

Per l'esame di TW valgono requisiti aggiuntivi di frontend, test e documentazione.

- La persistenza resta lato server; l'applicazione usa mappe interattive.
- Sono ammessi framework, librerie e strumenti anche diversi da quelli visti a lezione.
- Il deployment su hosting dedicato non è richiesto, ma l'applicazione deve essere rieseguibile in modo affidabile e semplice.

Deliverable richiesti

La consegna di TW comprende gli artefatti seguenti.

- SPA e backend conformi ai vincoli architetturali.
- **Documentazione dell'API**: elenco esaustivo degli endpoint e dei metodi supportati, generata automaticamente o prodotta manualmente.
- **Relazione sintetica** iniziale: ID del gruppo, componenti, tipologia di progetto (TW o TW + IS) e funzionalità assegnate.
- Almeno **10 test End-to-End (E2E)** automatici, con dati di esempio e istruzioni per eseguirli.

Svolgimento della discussione

La discussione di TW ha una struttura fissa in tre momenti.

- Il gruppo si presenta con l'applicazione già installata e funzionante sul proprio portatile.
- **Demo iniziale**: breve dimostrazione delle funzionalità, della responsiveness e dell'adattamento dei componenti al ridimensionamento del viewport.
- **Code review**: discussione di endpoint, scelte implementative e verifica del login per le funzionalità riservate.
- **Modifica live**: il docente chiede modifiche estemporanee (colore di un pulsante, inversione di elementi, opacità CSS, ordine dei componenti in mobile); il gruppo deve individuare subito le parti di codice da toccare e il tipo di modifica necessario.
- Se nessun membro ha un portatile, l'app può girare sul PC del docente solo con lo stack del corso (Node e framework tradizionali) oppure containerizzata con Docker Compose.

Criteri di valutazione

La valutazione di TW considera padronanza, correttezza e qualità complessive.

- **Padronanza** di tutto ciò che si consegna: ogni elemento inserito (animazioni, trasparenze, effetti CSS) è potenziale oggetto della modifica live.
- **Correttezza funzionale** e gestione completa di tutti i casi di errore.
- **Qualità del codice**: facilità di estensione, manutenibilità e uso corretto dei framework per SPA.
- **Semantica HTTP**: rispetto dei metodi e delle convenzioni REST.
- **Sicurezza, usabilità, accessibilità** (es. contrasto dei colori) e **responsiveness** del layout.
- **Lessico tecnico appropriato**: chiamare i concetti col nome giusto (component, callback), senza nomi improvvisati.
- L'interfaccia deve avere uno schema di colori coerente, una tipografia ragionevole e contrasto accessibile: è preferibile un'interfaccia semplice e curata a una piena di animazioni di cui non si è padroni.

Ingegneria del Software

L'esame di IS valuta l'intero ciclo di vita del software: il progetto è principalmente un **documento PDF**, e l'implementazione serve solo come prova di fattibilità del design e base per i test.

- L'analisi e il design vanno ragionati con soluzioni **object-oriented** e documentati con **UML**, eventualmente tramite strumenti CASE (es. Visual Paradigm).
- Con la consegna anticipata i documenti sono PDF separati; con quella finale un unico PDF complessivo.
- La consegna intermedia comprende solo la parte di analisi; nella consegna finale si può presentare una versione aggiornata dell'analisi, senza penalizzazione.

Analisi e specifica dei requisiti

Il primo blocco documentale è il documento SRS (Software Requirements Specification).

- **Glossario** dei termini specifici del dominio, per allineare la terminologia.
- **Use case diagram** relativi alle funzionalità assegnate.
- Individuazione degli **stakeholder**.
- **Requisiti non funzionali** quantificabili e verificabili in modo oggettivo e booleano: non formule come "bello, veloce, usabile".
- **Assunzioni esplicite** con cui il gruppo ha risolto ambiguità e lacune della traccia.
- Almeno **3 use case full-dressed** (1 solo in modalità anticipata), scelti dal gruppo tra i più significativi, cioè con dinamiche e condizioni d'errore non banali.
- **Mockup** dell'interfaccia grafica.
- Possono complementare l'analisi class diagram, sequence diagram o statechart di analisi.

System design

Il secondo blocco descrive e motiva l'architettura.

- Descrizione dell'architettura proposta con definizione esplicita di componenti, responsabilità e criteri di design.

- **Motivazione delle scelte** architetturali e tecnologiche, con analisi esplicita dei trade-off valutati: il ragionamento conta più della scelta in sé, e non basta scrivere "abbiamo usato client-server".
- Spiegazione del perché del linguaggio di implementazione e delle eventuali soluzioni alternative scartate.

Object design

Il terzo blocco scende nel dettaglio delle classi e della persistenza.

- **Schema della base dati**.
- **Class diagram** delle componenti applicative, accompagnato da schede **CRC** (Class-Responsibility-Collaboration) per le classi di progetto: non per classi di interfaccia grafica né per classi forzate dal linguaggio.
- **Sequence diagram** per gli use case full-dressed.
- Almeno uno **statechart** di un'entità con ciclo di vita non banale, scelta dal gruppo.
- Evidenze di Clean Code, Object-Oriented Design, **design pattern** e **dependency injection**, con spiegazione del problema che ciascuna scelta risolve.

Analisi statica e versioning

Il documento deve includere le evidenze di qualità del codice e del processo.

- **Report di analisi statica** (es. SonarQube / SonarCloud) con le criticità individuate e le relative soluzioni, oppure la motivazione della mancata risoluzione.
- **Evidenze del versioning Git**: contributor, frequenza dei commit e strategie di branching.
- Il codice non va inserito nel documento: l'implementazione si consegna come sorgente, non stampata.

Testing

Il testing si fa con framework di unit test, su argomenti ancora da affrontare a lezione.

- Suite di **test unitari** automatici (es. JUnit o XUnit), in numero variabile a seconda della modalità di consegna.
- Non esiste una soglia minima di coverage, ma la copertura ottenuta va raccontata e motivata in sede di consegna.

Presentazione del progetto

La discussione è la prova generale della presentazione della tesi di laurea.

- Durata: **10 minuti** di presentazione tecnica da gruppo di analisti maturi.
- **No marketing**: vietato spendere i primi minuti elencando i requisiti assegnati; il focus è sulle scelte ingegneristiche.
- **Design rationale**: spiegazione esplicita del perché delle scelte progettuali.
- Slide finale di **retrospettiva critica** (lessons learned): difficoltà principali, rework necessari e insegnamenti concreti del lavoro di squadra, senza banalità.
- Durante la presentazione si può aprire l'ambiente di sviluppo, principalmente per spiegare il design.

Pesi di valutazione

La valutazione di IS ripartisce i pesi così, con l'implementazione ridotta a prova di fattibilità.

Parte	Peso
Requisiti	20%
Design	30%
Testing	25%
Processo e teamwork	15%
Implementazione	10%

Warning

Overshooting

Un'implementazione estesissima accanto a una parte documentale povera è un fallimento: conviene lavorare in verticale su ciò che la traccia richiede. Le funzionalità extra non comportano penalizzazioni, ma nemmeno voti oltre il massimo, e allargano solo i tempi di realizzazione.

Uso dell'intelligenza artificiale

L'uso di AI generativa (es. ChatGPT, GitHub Copilot) è consentito e non rilevabile, ma è anacronistico vietarlo: chi lavorerà in azienda la userà quotidianamente.

Warning

Responsabilità individuale

Gli studenti sono totalmente ed esclusivamente responsabili di ogni riga di codice, diagramma o testo consegnato: in sede di orale i docenti verificano la profonda padronanza del lavoro, con richieste di giustificare qualsiasi scelta progettuale o sintattica e di apportare modifiche immediate.

- L'esperienza d'uso dell'AI, compresi limiti, allucinazioni e correzioni necessarie, può essere documentata proficuamente nella slide di retrospettiva (lessons learned).
- La stessa regola vale per TW sulla parte implementativa: bisogna saper identificare subito quale parte di codice modificare per cambiare un comportamento richiesto.